

Insight Agent

An Autonomous Data Analyst

An LLM agent that plans, writes and runs its own pandas / matplotlib code, self-corrects, and explains the result.

Built with LangGraph - LangChain - Groq / Gemini - Streamlit

Ankit Saxena

github.com/AnkitSaxena-AI

Technical Report - June 2026

Table of Contents

1. Introduction.....	3
2. What the Insight Agent Does.....	3
3. System Architecture.....	4
4. The Agent Loop (LangGraph).....	5
5. The Safe Code Executor.....	6
6. LLM Providers and Configuration.....	6
7. The Streamlit Application.....	7
8. Offline Testing and Validation.....	7
9. Running and Deploying.....	8
10. Limitations and Future Work.....	8
11. Conclusion.....	9
Appendix A: Interview and Viva Q&A.....	9

1. Introduction

Modern data teams spend a large share of their time on a repetitive loop: load a dataset, write a little code to slice it, look at the result, fix the inevitable error, and finally describe what the numbers mean. The Insight Agent automates that loop. It is a generative-AI application in which a large language model (LLM) acts as an autonomous analyst: given a CSV file and a plain-English question, it plans an approach, writes its own pandas and matplotlib code, executes that code against the data, reads the output or the traceback, corrects itself when it goes wrong, and finally returns a written answer alongside the chart and the exact code it ran.

This project was built to demonstrate practical, end-to-end competence with LLM agents and prompt engineering - specifically the LangChain and LangGraph ecosystem - rather than a simple wrapper around a chat model. The distinguishing feature is the closed feedback loop: the agent does not merely translate a question into a single query, it iterates the way a human analyst does in a notebook, observing results and repairing mistakes until the task is done or a retry budget is exhausted.

The remainder of this report describes what the system does, its architecture, the design of the safe code executor, the LangGraph reasoning loop, the configurable LLM back-end, the Streamlit interface, the offline test strategy that validates the whole pipeline without an API key, and the limitations and future directions of the work.

2. What the Insight Agent Does

From a user's perspective the workflow is simple. The user selects a dataset - either one of the bundled samples or their own uploaded CSV - types a question in natural language, and presses Analyze. The agent then carries out the following, all visible in the interface:

- **Plan:** it writes a short, concrete analysis plan in plain language, naming the columns and operations it intends to use.
- **Generate code:** it writes focused pandas / matplotlib code to carry out that plan.
- **Execute:** the code runs in a restricted sandbox against the user's DataFrame.
- **Observe and self-correct:** it reads the printed output, the returned value and any figures - or, on failure, the traceback - and rewrites the code if needed (up to a configurable number of attempts).
- **Synthesize:** it produces a concise, business-readable answer that cites the actual numbers, shown next to the chart and the final code.

Example questions the agent handles on the bundled data include "Which store has the highest average weekly sales?", "Do fiber-optic customers churn more than DSL customers?", and "What is the average price of waterfront versus non-waterfront homes?" Figure 1 shows a chart the agent wrote and executed itself when answering the first of these.

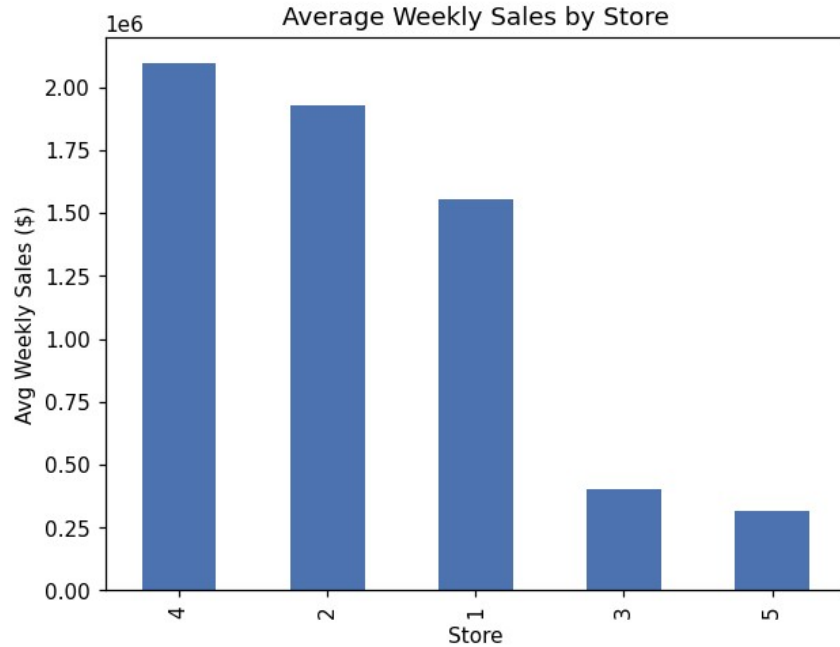


Figure 1: A chart produced autonomously by the agent for the question "Which store has the highest average weekly sales?"

3. System Architecture

The system has four cooperating parts: a thin Streamlit presentation layer; a LangGraph state machine that orchestrates the agent's reasoning; a safe executor that runs generated code and captures its effects; and a pluggable LLM provider. Figure 2 shows how a request flows through them.

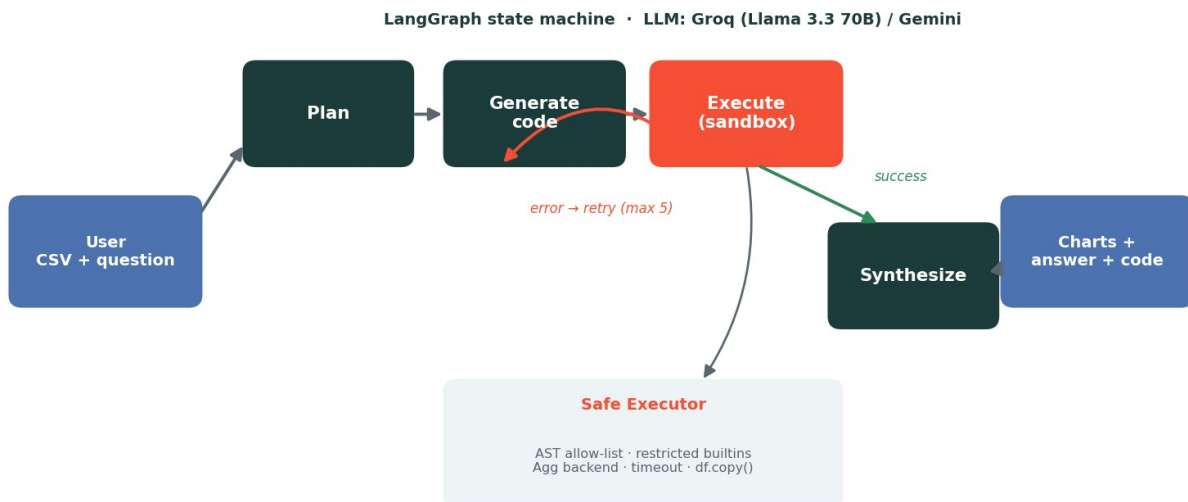


Figure 2: End-to-end architecture and the plan / generate / execute / synthesize loop.

The codebase is organised as a small, importable Python package so that the agent logic is fully decoupled from the user interface and can be tested in isolation:

```

insight_agent/
state.py      # AgentState: the shared blackboard between graph nodes
llm.py       # provider factory: Groq (default) / Gemini, lazy imports
prompts.py   # planner / coder / synthesizer prompts + schema summary
executor.py  # the safe Python executor (AST allow-list + sandbox)
graph.py     # the LangGraph state machine + run_agent()
app/app.py   # Streamlit UI
tests/test_offline.py # offline stub-LLM tests (no key / no network)

```

State is passed between nodes as a single typed dictionary. Keeping all working memory in one structure makes the agent transparent (the UI simply renders the trace that accumulates in it) and testable (a test can assert on any field of the final state). The fields are summarised below.

State field	Purpose
question / df / schema	The user's question, the DataFrame, and a text summary of its shape, dtypes and head used to ground the prompts.
plan	The planner's natural-language analysis plan.
code	The most recent block of generated code.
observation / last_error	What the agent saw after running its code; the traceback if it failed.
n_tries / max_tries	Execution counter and the retry budget that bounds the loop.
figures / answer	Captured charts (base64 PNG) and the final written analysis.
success / steps	Whether the last run succeeded, and the full step-by-step trace for the UI.

4. The Agent Loop (LangGraph)

The reasoning loop is implemented as an explicit LangGraph StateGraph rather than a single mega-prompt. LangGraph models an agent as a directed graph of nodes (functions that read and update the shared state) connected by edges, including conditional edges that branch on the state. This makes the control flow inspectable and the retry behaviour deterministic.

The graph has four nodes - plan, gen_code, execute and synthesize - wired as follows:

- plan to gen_code to execute run once in sequence on every request.
- After execute, a conditional edge inspects the state: if the run failed and the retry budget remains, it routes back to gen_code with the traceback attached so the model can repair its code; otherwise it proceeds to synthesize.
- synthesize produces the final answer and the graph terminates.

The conditional edge after execute is the heart of the agent - it is what turns a one-shot code generator into something that genuinely iterates. The retry budget (default five) guarantees the loop always terminates. The graph is built by a factory, build_graph(llm), that closes over an injected LLM; this single design choice is what makes the whole agent testable offline, because a test can pass a deterministic stub in place of a real model.

Each node attaches a record of its work to the steps list in the state, so the interface can later replay exactly what the agent did: the plan, every coding attempt, each execution result, and the final synthesis.

5. The Safe Code Executor

Allowing a language model to execute code it wrote is powerful but obviously hazardous, so the executor is the most carefully engineered component. It applies defence in depth: several independent layers, each of which would have to fail for unsafe code to run.

Layer	What it enforces
Static AST check	The code is parsed into an abstract syntax tree and rejected before execution if it imports anything outside an allow-list (pandas, numpy, matplotlib, scipy and similar), calls a banned function (open, eval, exec, <code>__import__</code> , <code>getattr</code> , ...), or touches dunder attributes such as <code>__globals__</code> - a classic sandbox-escape vector.
Restricted namespace	Approved code runs with a curated <code>__builtins__</code> whitelist and only df, pd, np and plt in scope, so it cannot reach the operating system or the network.
Headless backend	matplotlib is forced to the non-interactive Agg backend, which is thread-safe and needs no display; figures are captured straight to PNG.
Timeout	Code runs in a worker thread that is abandoned if it exceeds a configurable limit, so a runaway computation cannot hang the app.
Defensive copy	The agent receives <code>df.copy()</code> , so it can never corrupt the user's data between turns.

After a successful run the executor returns captured standard output, the repr of any trailing expression (REPL-style, so a bare `df.describe()` is shown), and every open figure as a base64-encoded PNG. On failure it returns the formatted traceback as text - which is precisely what is fed back into the next coding attempt. A short, structured observation string summarises all of this for the model.

It is worth being honest about scope: this is a pragmatic sandbox appropriate for a trusted, single-user analyst tool, not a hardened security boundary for arbitrary untrusted input. For a multi-tenant deployment the same executor would be wrapped in operating-system-level isolation such as a container with seccomp restrictions. The layered design makes that an extension rather than a rewrite.

6. LLM Providers and Configuration

The LLM is accessed through a small factory, `get_llm(provider, api_key)`, so the rest of the system is independent of any one vendor. Two providers are supported out of the box:

- Groq (default) - free, fast, no credit card required, serving the open Llama 3.3 70B model through an OpenAI-compatible API. Its very low latency makes the agent's step-by-step reasoning pleasant to watch live.
- Google Gemini (optional) - the gemini-1.5-flash model via langchain-google-genai, for users who prefer Google's free tier.

The provider is chosen by a dropdown in the app or the `LLM_PROVIDER` environment variable, and the relevant key is read from the environment or entered in the sidebar. Provider packages are imported lazily inside the factory, which keeps the default deployment lean (the Gemini dependency is only needed if Gemini is actually selected) and, importantly, means the offline test suite needs neither provider installed. API keys are never hard-coded: locally they live in a git-ignored `.env` file, and in deployment they are supplied through Streamlit secrets.

7. The Streamlit Application

The interface is a single Streamlit application. A sidebar collects configuration - provider, API key (entered as a password field), and advanced controls for the retry budget and per-execution timeout. The main pane walks the user through three steps: choose data, ask a question, read the answer.

- **Data:** the user picks a bundled sample or uploads a CSV; a live preview shows the first rows and the shape.
- **Question:** a free-text box, with one-click suggested questions tailored to each bundled dataset.
- **Answer:** the synthesized response is shown first, followed by the generated charts, then an expandable agent trace that reveals the plan, every code attempt, each execution result, and finally the exact code that produced the answer.

Exposing the full trace is a deliberate choice. It turns the agent from an opaque oracle into a transparent collaborator: the user can see the reasoning, verify the code, and trust (or challenge) the result - which is exactly the posture a data professional should take toward an automated analyst.

8. Offline Testing and Validation

Because live LLM endpoints are not always reachable (and to keep the test suite fast, free and deterministic), the entire pipeline is validated offline using a stub LLM - a tiny deterministic object that returns canned responses in place of a real model. This proves the control flow, the executor and the self-correction loop without any API key or network access.

The headline test exercises the complete LangGraph agent end-to-end. The stub is deliberately adversarial: on its first coding turn it emits broken code (a deliberate column-name typo that raises a `KeyError`); only when it sees the words "PREVIOUS CODE FAILED" in the retry prompt does it return corrected code. The test then asserts that the agent recovered on its own. The suite covers:

Test	What it proves
Executor runs and captures a figure	Generated code executes, prints output, and a matplotlib chart is captured as PNG.
Executor reports errors as text	A failing snippet yields a <code>KeyError</code> traceback as a string rather than crashing the app.
Safety blocks dangerous code	<code>import os</code> , <code>open()</code> , <code>__import__</code> , dunder access and <code>eval</code> are all

Test	What it proves
	rejected before running.
Safety allows normal analysis	Legitimate numpy / pandas code passes the static check.
Graph self-corrects and synthesizes	The full agent fails once, retries (n_tries = 2), captures a chart, and produces an answer; the trace visits plan, two gen_code and two execute steps, then synthesize.

All six tests pass in roughly two seconds. The key assertion - that the agent's first execution fails and its second succeeds with no human intervention - is the concrete proof that the self-correction loop works as designed.

```
$ pytest -q
..... [100%]
6 passed in 1.78s
```

9. Running and Deploying

Running the project locally takes a few minutes. After cloning the repository and installing the lean runtime requirements, the user obtains a free Groq key, places it in a .env file, and launches the app:

```
pip install -r requirements.txt
cp .env.example .env # paste your free Groq key
streamlit run app/app.py
```

The offline tests need no key: pip install -r requirements-dev.txt then pytest -q. For a public demo, the app deploys to Streamlit Community Cloud directly from GitHub; the API key is provided through the platform's encrypted Secrets rather than committed to the repository. A deliberate lesson carried over from earlier projects is to keep requirements.txt lean - only the app's true runtime dependencies - because bloated requirements make free-tier deployments slow or unreliable. Heavy or optional dependencies live in separate requirements files.

10. Limitations and Future Work

The system has clear, understood limits. The quality of the analysis depends on the chosen LLM; very wide or messy CSVs may require one or two extra retries, and the retry budget intentionally caps how hard the agent tries before giving its best effort. The executor, as discussed, is a pragmatic sandbox rather than a security boundary for untrusted users. Free LLM tiers are rate-limited, so heavy use can hit quotas.

Natural extensions include: multi-step analyses that carry a result from one question into the next; support for multiple files and joins; a wider chart vocabulary and optional interactive (Plotly) figures; exporting a full analysis as a notebook or PDF; OS-level sandboxing for untrusted deployment; and a caching layer so repeated questions on the same data are instant. None of these require changing the core loop - they extend it.

11. Conclusion

The Insight Agent demonstrates a complete, working LLM agent that does real data science: it plans, writes and runs its own code, recovers from its own mistakes, and explains its findings - all behind a clean, transparent interface and on a swappable, free LLM back-end. It is engineered the way production software should be: a decoupled package, a carefully layered safe executor, an explicit and inspectable LangGraph control flow, and an offline test suite that proves the hardest behaviour (self-correction) without a single API call. As a portfolio piece it directly evidences the LangChain, LangGraph and prompt-engineering skills it sets out to show, and it does so on a genuinely useful, non-trivial application rather than a toy chatbot.

Appendix A: Interview and Viva Q&A

A selection of questions an interviewer might ask about this project, with concise answers.

Q. What makes this an "agent" rather than just a prompt?

An agent perceives, decides, acts and observes in a loop, using tools and adapting to results. Here the LLM plans, then takes an action (writing and running code) through a tool (the executor), observes the outcome, and decides whether to retry or finish. A plain prompt would generate one answer with no execution and no feedback; this system closes that loop.

Q. Why LangGraph instead of a simple while-loop or a LangChain chain?

LangGraph models the agent as an explicit state machine with conditional edges, which makes the control flow - especially the retry branch - inspectable, deterministic and easy to extend. A hand-rolled loop would work but would scatter state management and routing logic through the code. A linear LangChain chain cannot express the conditional "retry on error" edge cleanly. LangGraph is also the industry-standard way to build this pattern, which is part of the point of the project.

Q. How does the self-correction actually work?

When generated code fails, the executor returns the traceback as text. The routing function sees success is false and, if retries remain, sends control back to the code-generation node. That node detects the prior error in the state and includes the failed code and its traceback in the next prompt, asking the model to diagnose and fix it. The loop repeats until the code runs or the retry budget is spent.

Q. How do you stop the model from running malicious or destructive code?

Defence in depth. Before any execution, the code is parsed into an AST and rejected if it imports outside an allow-list, calls dangerous builtins like open or eval, or accesses dunder attributes used for sandbox escapes. Approved code then runs with a restricted set of builtins and only the DataFrame and core libraries in scope, on a headless matplotlib backend, under a timeout, against a copy of the data. I'm also clear that this is a trusted-user sandbox, not a boundary for arbitrary untrusted input - for that I'd add container-level isolation.

Q. How is this different from Retrieval-Augmented Generation (RAG)?

RAG retrieves relevant text and conditions the model's answer on it - it is about grounding language in documents. This agent does not retrieve text; it writes and executes code to compute

answers from structured data. They solve different problems: RAG for unstructured knowledge, this for quantitative analysis. The GraphRAG project planned next is the RAG-style counterpart.

Q. How do you keep the model from inventing column names?

Every prompt is grounded with a schema summary generated from the actual DataFrame: shape, each column's name, dtype, non-null count and example values, plus `df.head()`. Giving the model the real columns up front sharply reduces hallucinated names, and when one slips through, the resulting `KeyError` is caught and fed back for correction.

Q. Why support two LLM providers, and how is it implemented?

Vendor flexibility and resilience: free tiers and models change, so a swappable back-end avoids lock-in and is a small but real engineering signal. It is implemented as a factory function that returns the right LangChain chat model based on a provider argument or environment variable, importing each provider's package lazily so the default install stays lean and the tests need neither.

Q. How did you test something that depends on a non-deterministic LLM?

By injecting a stub. The graph is built by a factory that takes the LLM as an argument, so tests pass a deterministic object that returns canned plans and code. That removes the non-determinism and the network entirely, and lets me script an adversarial case - broken code first, fixed code on retry - to prove the self-correction path. The real model is exercised manually at run time.

Q. What would you change to make this production-ready?

Run the executor inside an isolated container with strict resource and syscall limits; add authentication, request quotas and observability (logging, tracing, cost tracking); cache results per dataset and question; stream the agent's steps to the UI token-by-token; and add evaluation harnesses that score answer correctness on a labelled question set so prompt or model changes can be measured rather than guessed.

Q. What was the hardest part to get right?

The executor. Capturing standard output, a trailing expression value and matplotlib figures together, while enforcing the AST allow-list and a cross-platform timeout, took the most care - and it is also the component most central to safety, so it carries the most tests.